



Diagrama de classes

Introdução

Nesta aula, vamos explorar um dos diagramas mais importantes da UML: o diagrama de classes. Ele é fundamental para representar a estrutura estática de um sistema de software, permitindo que visualizemos as classes, seus atributos, métodos e os relacionamentos entre elas. Iniciaremos com a definição dos principais componentes das classes, como atributos e métodos, e avançaremos para conceitos mais complexos, como composição, agregação e multiplicidade. Ao final, discutiremos a visibilidade, essencial para o controle de acesso a informações dentro de um sistema orientado a objetos.

Componentes do diagrama de classes

Vamos abordar os componentes essenciais do diagrama de classes, um dos principais diagramas da linguagem UML (Unified Modeling Language). A UML facilita a visualização de sistemas complexos, permitindo a criação de representações gráficas que ajudam na compreensão das decisões tomadas durante a análise de um projeto. Essa modelagem não só melhora a comunicação entre a equipe, mas também elimina ambiguidades, o que torna a interpretação do sistema mais clara. Em termos de especificação, a UML oferece modelos completos e precisos, essenciais para as diferentes fases do desenvolvimento de um sistema, independentemente da metodologia ou tecnologia utilizada. Além disso, a UML também permite a integração com linguagens de programação, possibilitando a geração de código a partir dos diagramas ou, inversamente, a criação de diagramas a partir do código existente.

Os diagramas de classe, um dos principais componentes da UML, são usados para modelar a estrutura estática de um sistema. Eles descrevem as classes que compõem o sistema, os atributos de cada classe, seus métodos e os relacionamentos entre essas classes. Um diagrama de classes não apenas modela as classes em si, mas também os objetos do sistema, suas interações e os serviços que oferecem. Em resumo, os diagramas de classe focam na estrutura estática de um sistema, destacando os serviços que cada classe fornece, independentemente de solicitações de outras classes.

Entre os principais elementos representados nos diagramas de classes estão as classes, os objetos e os pacotes. As classes são representações abstratas de conjuntos de objetos que compartilham a mesma estrutura (atributos) e comportamento (métodos). Por outro lado, os objetos são instâncias dessas classes. Os pacotes agrupam elementos relacionados do modelo, organizando-os de forma hierarquizada para facilitar a compreensão. Essa organização é especialmente útil em sistemas complexos, onde é necessário visualizar camadas e subsistemas de maneira clara.

Um exemplo prático é a classe “Carro.” Essa classe teria atributos como marca, modelo, cor, ano e número de portas, e métodos como ligar, desligar, acelerar e frear. No diagrama, a classe seria representada por um retângulo dividido em três partes: o nome da classe, os atributos e os métodos. Por exemplo, o atributo “modeloCarroceria” seria escrito em camelCase, ou seja, sem espaços, com a segunda palavra iniciando com letra maiúscula.

Outro exemplo é a classe “Moto,” que também teria seus próprios atributos e métodos, seguindo o padrão UML. A representação gráfica de uma classe vai além de um simples retângulo com três divisões. Esses diagramas podem ser complementados por outros elementos, que serão abordados mais adiante.

Composição

A composição é um dos conceitos centrais da modelagem orientada a objetos e está diretamente relacionada ao processo de decomposição de sistemas. Quando enfrentamos um problema complexo, podemos dividi-lo em partes menores, tornando mais fácil sua compreensão e resolução. Um dos três tipos principais de relacionamento que surgem nesse processo de decomposição é a composição, ao lado da associação e da agregação. No entanto, enquanto a decomposição foca em quebrar um problema em partes menores, a composição lida com a construção de um todo a partir dessas partes menores, onde a existência das partes depende da existência do todo.

A composição em UML representa uma relação de dependência forte entre os objetos. Ou seja, as partes que compõem o sistema não existem independentemente do todo. Um exemplo clássico seria a relação entre uma casa e um banheiro. A casa pode existir sem um banheiro, dependendo do cenário, mas o banheiro não existe sem a casa, pois faz parte dela. Assim, se o objeto da classe “Casa” for eliminado, o objeto “Banheiro” associado também deixa de existir. Esse tipo de relação é representado por um losango preenchido na UML, indicando que há uma composição entre os dois objetos.

Outro exemplo prático é a relação entre uma árvore e suas folhas. A classe “Árvore” possui atributos como altura e idade, enquanto a classe “Folha” contém características como cor e tamanho. As folhas, nesse caso, compõem a árvore e não podem existir fora dela. A árvore é um conjunto de folhas, e a existência das folhas depende da árvore, caracterizando uma relação de composição.

Podemos ainda observar o conceito de composição na relação entre uma escola e suas salas de aula. A classe “Escola” contém um conjunto de salas,

representadas por um array. No entanto, uma sala de aula não existe fora do contexto da escola. Se o objeto da classe “Escola” for eliminado, todas as suas salas associadas também deixarão de existir, reforçando o conceito de contenção e dependência.

Por fim, um exemplo mais complexo seria a classe “Pessoa” que é composta por órgãos como coração e pulmões. A pessoa, nesse caso, não pode existir sem esses órgãos, e esses órgãos também não podem existir sem o contexto da pessoa. A classe “Coração” e a classe “Pulmão” são parte essencial da pessoa, e sua existência depende da classe “Pessoa”. Esse é mais um exemplo que ilustra a importância da composição em sistemas orientados a objetos.

Agregação

A agregação é um tipo de relacionamento que representa uma associação entre o todo e as partes, porém de maneira mais fraca do que a composição. Enquanto na composição há uma dependência forte entre o todo e suas partes, na agregação a relação é mais flexível. As partes podem existir de maneira independente do todo, ou seja, o ciclo de vida das partes não está necessariamente atrelado ao ciclo de vida do todo.

Na agregação, as partes têm uma vida própria e podem ser criadas ou destruídas independentemente do todo. Um exemplo prático desse tipo de relação é o relacionamento entre uma casa e um móvel. Ao contrário da relação entre uma casa e um banheiro (que caracteriza uma composição), o móvel pode existir sem a casa. A casa pode agregar diversos móveis, mas os móveis podem ser usados em outras casas ou até mesmo existirem sozinhos. O ciclo de vida de um móvel é, portanto, independente do ciclo de vida da casa.

Outro exemplo que ilustra a agregação é o relacionamento entre uma escola e seus professores. Na composição, como no caso da escola e as salas de

aula, as salas de aula dependem da existência da escola para existir. Já na agregação, o professor pode estar associado a uma ou várias escolas, mas sua existência independe da escola. O ciclo de vida da escola e do professor são completamente separados, reforçando a ideia de que essa é uma relação mais fraca do que a composição.

Um terceiro exemplo que podemos citar é o relacionamento entre uma loja e seus produtos. A loja pode conter uma lista de produtos, mas esses produtos podem existir independentemente da loja. Por exemplo, se a loja deixar de existir, os produtos ainda podem ser vendidos em outras lojas ou continuar a existir fora do contexto da loja. O ciclo de vida do produto é autônomo e separado do ciclo de vida da loja, evidenciando a relação de agregação.

Dessa forma, podemos concluir que a agregação representa uma relação onde o todo agrega as partes, mas essas partes não são dependentes do todo para existir. Esse tipo de relacionamento é útil para modelar situações em que os componentes podem existir de forma independente, sem criar uma forte dependência como na composição.

Multiplicidade

A multiplicidade é uma propriedade fundamental nas associações entre diferentes classes em um diagrama de classes. Ela define quantas instâncias de uma classe podem estar associadas a instâncias de outra classe. Em outras palavras, a multiplicidade especifica o número mínimo e máximo de objetos que podem participar de uma associação, ajudando a esclarecer a relação entre as classes envolvidas.

Vamos analisar um exemplo com as classes “Casa” e “Móvel”. Nesse caso, temos um relacionamento do tipo agregação, em que uma casa pode agregar diversos móveis, mas os móveis existem independentemente da

casa. A multiplicidade pode ser representada pelo número "0...", *indicando que uma casa pode ter nenhum ou muitos móveis. Da mesma forma, o móvel pode estar associado a zero ou muitas casas. Isso caracteriza uma relação flexível, onde os objetos podem ou não estar relacionados entre si.*

Agora, vejamos o relacionamento entre as classes "Casa" e "Banheiro", que é um exemplo de composição. Uma casa pode ter um ou muitos banheiros, mas um banheiro necessariamente faz parte de uma casa. Na multiplicidade, isso é representado por "1" ao lado da classe "Banheiro", indicando que o banheiro não existe sem a casa. No entanto, do lado da casa, podemos ter "0...", o que significa que uma casa pode ter nenhum ou vários banheiros, dependendo do cenário.

Outro exemplo relevante é a relação entre as classes "Escola" e "Professor". Aqui, estamos lidando com uma agregação, onde uma escola pode ter nenhum, um ou vários professores associados a ela. Do outro lado, um professor pode estar vinculado a uma ou mais escolas, ou até mesmo não estar associado a nenhuma. Esse tipo de multiplicidade é representado por "0..." *em ambos os lados da relação, mostrando a flexibilidade no número de associações.*

Já no caso de uma árvore e suas folhas, temos um relacionamento de composição. Uma árvore pode ter uma ou muitas folhas, ou estar completamente desfolhada. Por outro lado, uma folha sempre pertence a uma única árvore. Aqui, a multiplicidade seria "1" para a árvore e "0..." para as folhas, demonstrando que a folha está obrigatoriamente associada a uma árvore, mas a árvore pode ter várias folhas ou nenhuma.

Por fim, analisemos a relação entre a classe "Pessoa" e os órgãos "Coração" e "Pulmão". Uma pessoa pode ter no máximo um coração, e necessariamente tem que ter um, o que resulta em uma multiplicidade de "1..1" para essa associação. Já no caso dos pulmões, uma pessoa pode ter um ou dois pulmões, o que podemos representar com a multiplicidade "1..

2". Cada pulmão pertence a uma única pessoa, reforçando a ideia de uma relação de composição com multiplicidade específica.

A multiplicidade, portanto, é uma ferramenta essencial para modelar relações entre classes, especificando com clareza o número de instâncias que podem estar associadas entre si, seja em relações de agregação ou composição.

Visibilidade

A visibilidade em um diagrama de classes é fundamental para definir quais atributos e métodos de uma classe podem ser acessados por outras classes. Ela está diretamente relacionada ao conceito de encapsulamento, que visa ocultar os detalhes internos de uma classe, expondo apenas o que é necessário. A escolha correta da visibilidade é um dos pilares da programação orientada a objetos, pois garante a segurança e a manutenibilidade do código, limitando o acesso a determinados elementos.

Existem quatro tipos principais de visibilidade. O primeiro é o privado, representado por um símbolo de menos (-). Elementos privados só podem ser acessados dentro da própria classe, ou seja, nenhum outro objeto pode alterar ou visualizar um atributo privado. Por exemplo, se uma classe "Carro" possui um atributo "cor" privado, nenhuma outra classe poderá acessar ou modificar esse atributo diretamente.

O segundo tipo de visibilidade é o público, representado pelo símbolo de mais (+). Elementos públicos podem ser acessados por qualquer classe. No mesmo exemplo da classe "Carro", se o método "acelerar" for público, ele pode ser utilizado por qualquer outra classe. Esse tipo de visibilidade é essencial para os métodos que precisam ser chamados por outros objetos, enquanto os atributos geralmente devem permanecer privados para manter o encapsulamento.

A visibilidade protegida, representada por um símbolo de suspenso (#), permite que os elementos sejam acessados pela própria classe e suas subclasses. Isso é útil quando você deseja que as classes derivadas tenham acesso a certos atributos ou métodos sem que eles estejam visíveis para todas as outras classes. Por exemplo, em uma classe “Cachorro”, o método “correr” pode ser protegido, permitindo que subclasses como “CachorroDeCorrida” acessem esse método, mas impedindo o acesso de outras classes.

Por fim, temos a visibilidade de pacote, simbolizada por um til (~). Esse tipo de visibilidade permite que os atributos e métodos sejam acessados por todas as classes dentro do mesmo pacote. Isso é útil para manter a comunicação entre classes que fazem parte de um mesmo grupo, sem expor esses elementos para todo o sistema.

Vamos ver um exemplo prático: considere a classe “ContaBancária” com os atributos “saldo”, “titular” e “número da conta”. Se o “saldo” for público, ele poderá ser acessado por outras classes. Já o “titular” pode ser privado, protegendo a identidade do titular, e o “número da conta” pode ser protegido, permitindo que classes derivadas da “ContaBancária” tenham acesso a ele.

Concluimos nossa aula sobre diagrama de classes destacando a importância dessa ferramenta na modelagem de sistemas orientados a objetos. Aprendemos a identificar os componentes principais, como classes, atributos e métodos, além de explorar as relações entre classes por meio dos conceitos de composição, agregação e multiplicidade. Finalizamos com o entendimento da visibilidade, fundamental para o controle de acesso e proteção dos dados em um sistema.

Conteúdo Bônus

Para se aprofundar no tema desta aula, sugiro que assista ao vídeo intitulado “Tutorial de Diagramas de Classes UML”, que está disponível no canal Lucid Software Português no YouTube.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. **Fundamentos da programação de computadores**. 3.ed. Pearson: 2012.

BRAGA, P. H. **Teste de software**. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). **Arquitetura de software**. Pearson: 2017.

GALLOTTI, G. M. A. **Qualidade de software**. Pearson: 2015.

MEDEIROS, E. **Desenvolvendo software com UML 2.0 definitivo**. Pearson: 2004.

MORAIS, I. S. de. (Org.). **Engenharia de software**. Pearson: 2017.

PFLEEGER, S. L. **Engenharia de software: teoria e prática**. 2.ed. Pearson: 2003.

SOMMERVILLE, I. **Engenharia de software**. 10.ed. Pearson: 2019.

Ir para exercício